Harshini Lakshman
Amanda Gbe
Daniel Yu

COMPSCI 1XD3 Development Plan Report

Date: March 11th, 2026

Professor: Dr. Sam Scott

Meet the Team

Harshini Lakshman, Amanda Gbe, and Daniel Yu are working together under the team name *PixelForge*. The client is Sophie Xiu, whose affiliation is in the Computer Science Society (CSS) as an executive team member. The client's request is to create a web application that is a central informational hub for CSS members and those interested in the society. The working name of this application is Computer Science Society Web Hub.

Web Application Description

The purpose of this web application is to have one central location to find all information related to the society for members and prospective members of the Computer Science Society (CSS). Currently, CSS information is spread across multiple platforms, making it difficult to stay organized. This application will improve accessibility, organization and user engagement by bringing all features together into a single system.

The application will include a login system where users sign in using their MacID. Once logged in, users will have access to personalized features connected to their account.

Key features of the application include:

- A calendar display upcoming and past CSS events, along with links to livestreams or recorded sessions (eg Youtube review sessions)
- An academic resources page containing notes, review slides and solutions to practice questions.
- Forms that allow users to apply for executive positions, request new events and submit merchandise orders
- A merchandise systems where users can request items and track their order status
- A contact section with links to CSS social media platforms (Instagram, Youtube, Discord) and executive contact information

In addition, users will be able to interact with the platform by submitting event suggestions, completing forms and accessing resources, making the application both informative and interactive.

Overall, this web application will create a more connected and engaging environment for students by improving access to information, supporting academic success, and increasing participation in CSS activities.

Data Description

Table 1: users

Why we need it: This table stores the account information for each student so the system can identify who is logged in and connect them to their event, forms, quizzes and activity.

Fields	Data Types	
email	VARCHAR(100)	<= Primary Key
mac_id	VARCHAR(50)	
first_name	VARCHAR(50)	
last_name	VARCHAR(50)	
password_hash	VARCHAR(255)	

Using email instead matches more to real world systems (like google, mosaic) and makes the project more professional. Also, simplifies user identification across all tables, reduces backend complexity and allows proper validation of McMaster students making the application more scalable and user-friendly.

Table 2: events

Why we need it: This table stores all CSS events so users can view upcoming and past events on the calendar.
What it stores: It stores event details such as the title, date, time, location and description.

Fields	Data Types	
event_id	INT	<= Primary Key
event_title	VARCHAR(100)	
event_description	TEXT	
event_date	DATE	
event_time	TIME	
location	VARCHAR(100)	
stream_link	VARCHAR(255)	

Table 3: event_registrations

Why we need it: This table links students to the events that they registered for so their personal calendar can be shown after login.
What it stores: It stores which student registered for which event.

Fields	Data Types	
registration_id	INT	<= Primary Key

email	VARCHAR(100)
event_id	INT
registration_date	DATETIME

Table 4: resources

Why we need it: This table stores academic resources so users can access study materials in one place.
What it stores: It stores notes, review slides, answered questions, and links to academic help materials

Fields	**Data Types**	
resource_id	INT	<= Primary Key
resource_title	VARCHAR(100)	
resource_type	VARCHAR(50)	
resource_description	TEXT	
file_link	VARCHAR(255)	
course_code	VARCHAR(20)	
upload_date	DATE	

Table 5: resource_usage

Why we need it: This table tracks which academic resources a student has accessed so the app can show resources they have used.
What it stores: It stores links between students and the resources they viewed or downloaded.

Fields	**Data Types**	
usage_id	INT	<= Primary Key
email	VARCHAR(100)	
resource_id	INT	
date_accessed	DATETIME	

Table 6: forms

Why we need it: This table stores the different forms available in the app, such as joining exec, merchandise orders, or event requests.

What it stores: It stores general information about each form.

Fields	Data Types	
form_id	INT	<= Primary Key
form_title	VARCHAR(100)	
form_type	VARCHAR(50)	
form_description	TEXT	
create_date	DATE	

Table 7: form_questions

Why we need it: This table stores the questions belonging to each form so each form can have its own set of inputs.
What it stores: It stores the text, type and order of questions for each form.

Fields	Data Types	
question_id	INT	<= Primary Key
form_id	INT	
question_text	TEXT	
question_type	VARCHAR(50)	
question_order	INT	

Table 8: form_responses

Why we need it: This table stores each student's answers to forms they complete.
What it stores: it stores which student answered which form question and their response.

Fields	Data Types	
response_id	INT	<= Primary Key
email	VARCHAR(100)	
form_id	INT	
question_id	INT	

response_text	TEXT
submitted_at	DATETIME

Table 9: event_suggestions

Why we need it: This table stores event suggestions or questions submitted by users.
What it stores: It stores student-submitted ideas and questions for possible future CSS events.

Fields	**Data Types**	
suggestion_id	INT	<= Primary Key
email	VARCHAR(100)	
subject	VARCHAR(100)	
message	TEXT	
submitted_at	DATETIME	

Table 10: merch_orders

Why we need it: This table stores merchandise requests submitted by users.
What it stores: It stores order details for CSS merchandise without processing payment online.

Fields	**Data Types**	
order_id	INT	<= Primary Key
email	VARCHAR(100)	
item_name	VARCHAR(100)	
size	VARCHAR(20)	
quantity	INT	
order_date	DATETIME	
status	VARCHAR(50)	

Functions Performed by the Application

4.1 Viewing CSS Information

The application is to allow users to view general information about the Computer Science Society (CSS) including details about the executive team, links to social media, contact information, and event listings.

- Database effect: This function will use SELECT operations on the events table and the resources table to retrieve and display information.
- Client-side functionality: HTML will structure the content, CSS will style the page, and JavaScript may be used to improve navigation and interactive display elements.

4.2 View Upcoming and Past Events

The application will allow users to browse upcoming and past CSS events.

- Database effects: This function will use SELECT operations on the events table to retrieve event title, description, date, time, location and stream link.
- Client-side functionality: Events will be displayed using HTML cards or lists, CSS styling and responsive layouts. JavaScript may be used for filtering or dynamically displaying event details.
- Filter events: Users can filter available events by title, location, or date. Both list view and calendar view update dynamically without page reload. (Effect: client-side JavaScript filtering of existing SELECT data.)
- Toggle between list and calendar view: Users can switch between a traditional event list and a month-grid calendar.

4.3 Log In to an Account

The application is to allow a student to log in using their email and password.

- Database effect: This function will use SELECT operations on the users table to verify that the entered ~~student_id~~ email and password match an existing account.
- Client-side functionality: HTML forms will collect login credentials, Javascript will validate that required fields are completed and PHP will process the login request.

4.3.2 Create an Account

The application is to allow a new user to create an account using their mcmaster email, names, password and password confirmation. This allows versatility, allowing new users of the app.

- Database effect:
    - SELECT operations on the user table to check if email already exists with another account.
    - INSERT operations on the user table to add a new user record with the hashed password, first name, last name, email and generated mac_id.
- Client-side functionality: HTML forms collect user registration detail and PHP processes registration requests

4.4 Viewing Personalized Dashboard

After logging in, the applications will allow a student to view a personalized dashboard showing registered events, completed forms, submitted suggestions, merchandise orders and access to academic resources.

- Database effect: This function will use SELECT operations on the events_registration, form_responses, event_suggestions, merch_orders and resource_usage tables, filtered by the user's ~~student_id~~ email.
- Client_side functionality: HTML will organize dashboard sections, CSS will style the layout and Javascript may be used to expand, hide or dynamically load information.
- Users will be able to select a form from a dropdown menu with a list populated from the database, then load corresponding questions from the form using AJAX.

4.5 Register for an Event

The application will allow a logged-in student or register for a CSS event.

- Database effect: This function will use an INSERT operation on the event_registrations table to store the relationship between ~~student_id~~ email and event_id, along with the registration date.
- Client_side functionality: HTML buttons or forms will allow the user to register, JavaScript may confirm the action or prevent duplicate submissions and PHP will process the registration.
- Uses AJAX on the register button to dynamically register the user for the event without a full page refresh.
- Unregister from an event: Loggedin users can remove themselves from a previously registered event via a modal confirmation. (Effect: DELETE on event_registrations table.)

4.5.2 Unregister for an event

The application is to allow a student to unregister from a previously registered event. This accommodates users' changes if the user is no longer able to attend an event making the website more user friendly.

- Database effect: DELETE operations on the events_registration table to remove the user's event
- Client-side functionality: JavaScript for pop up confirmation and updating the button. PHP processes the request.

4.6 Viewing Registered Events on Personalized Calendar

The application will allow a logged-in student to view events they have registered for on a personal calendar or schedule page.

- Database effect: This function will use SELECT operations on the events_registrations table and the events table to retrieve the events of the logged-in user.
- Client_side functionality: HTML will structure the event schedule and CSS will style it. The events section displays all the events that the user is currently registered for and filters them by date.

4.7 Access Academic Resources

The application will allow users to browse academic resources such as notes, review slides, answered questions and study materials.

- Database effect: This function will use SELECT operations on the resources table to retrieve the available academic resources.
- Client_side functionality: HTML links, cards or lists will display resources, CSS will improve readability and JavaScript may be used for sorting, filtering or search.

4.8 Record Academic Resource Usage

The application will track which resources a logged-in student has accessed.

- Database effect: This function will use an INSERT operation on the resources_usage table whenever a student opens or downloads a resource.
- Client_side functionality: JavaScript may trigger the tracking action when a resource is clicked, while PHP stores the usage information in the database.

4.9 Complete Forms

The application will allow users to complete forms such as event request forms, applications or other CSS forms.

- Database effect:
    - SELECT operations will retrieve form details from the forms table and form questions from the form_questions table.
    - INSERT operations will store user responses in the form_responses table.
- Client_side functionality: HTML forms will display questions, JavaScript will validate required fields and input formats and PHP will process and store the submitted responses.

4.10 Submit Event Suggestions or Questions

The application will allow users to submit suggestions or future events or questions related to review sessions and CSS activities.

- Database effect:This function will use an INSERT operation on the event_suggestions table to store the submitted subject, message, student ID and submission time.
- Client_side functionality: HTML form will collect the suggestion or question, JavaScript will validate the form before submission and PHP will process the request and store it in the database.

4.11 Submit Merchandise Orders

The application will allow students to submit merchandise orders by selecting an item, size and quality.

- Database effect:
    - INSERT operation will store the order in the merch_orders table with an initial status of “pending”.
    - UPDATE operation will later modify the status field from pending → paid → completed once payment is confirmed.
- Client_side functionality: HTML forms will collect order information, JavaScript will validate inputs (e.g. required fields, quantity) and PHP will process and store the order.

4.12 View Merchandise Order Status

The application will allow a logged-in student to view the status of merchandise orders they have submitted.

- Database effect:. This function will use the SELECT operation on the merch_orders table filtered by ~~student_id~~ email.
- Client_side functionality: HTML will display the student's orders, CSS will style the order history and JavaScript may be used for filtering or sorting.

4.13 Responsive and Interactive User Interface

The application will include interactive and responsive user interface features to improve usability and accessibility.

- Database effect: This function has no direct database effect.
- Client_side functionality: HTML will provide the structure of the pages, CSS will implement styling, layout, hover effects and mobile responsiveness and JavaScript will provide validation, feedback messages and interactive features such as buttons, menus and dynamic content updates

4.14 Admin

The application allows a registered admin to update the status of merchandise orders, reject event suggestions and see help messages submitted by users.

4.14.1 - Orders

- Database effect:
    - SELECT operations on the merch_orders table to retrieve all submitted orders.
    - UPDATE operations to modify order statuses (pending, paid, shipped)
- Client-side functionality: HTML will display a structured list or table of all orders including item name, quantity, size and current status. CSS used to style the admin tables and change colors based on order status.
- Sends an AJAX request to update the order status of a merchandise

4.14.2 - Event Suggestions

- Database effect: UPDATE to approve (set status to 'approved') or DELETE to reject (remove suggestion).
- Client-side functionality: HTML will display a structured list or table of all suggestions with a reject button beside them. CSS used to style the admin tables and change colors based on order status

4.14.3 - Help Messages

- Database effect:
    - INSERT operation on the help_messages table to store submitted messages including user's email, subject, message content and timestamp.
    - SELECT operation will be used to retrieve and display all submitted messages, ordered by submission date

- Client-side functionality: HTML will provide a structured form for users to enter a subject and message, along with links to external platforms. CSS will style and link. JavaScript may be used for form validation to ensure all required fields.

4.14.4 - Contact page

- Database effect: None.
- Client-side functionality: Static HTML page with links to CSS social media (Instagram, YouTube, Discord).

Roles and Responsibilities

AMANDA - User Accounts & Events

Sections 4.1, 4.2, 4.3, 4.5, 4.6, 4.14.3

Responsible for login systems, viewing events, event registration

- [ ] HTML/CSS: Build and style login pages, event listings
- [ ] JavaScript: Validate login and handle event interaction
- [ ] PHP: Authenticate users and process event registration
- [ ] Database: Use users, events, event_registrations (SELECT, INSERT, DELETE)

HARSHINI - Forms & Suggestions

Section 4.4, 4.9, 4.10

Responsible for forms, event suggestions and dashboard (forms-related data).

- [ ] HTML/CSS: Build and style forms and dashboard
- [ ] JavaScript: Validate form inputs and provide feedback
- [ ] PHP: Process form submissions and suggestions
- [ ] Database: Use forms, form_questions, form_responses, event_suggestions (SELECT, INSERT)

DANIEL - Resources, Merchandise, Admin, Contact page

Section 4.7, 4.8, 4.11, 4.12, 4.14.1, 4.14.2

Responsible for academic resources, merchandise orders and index page.

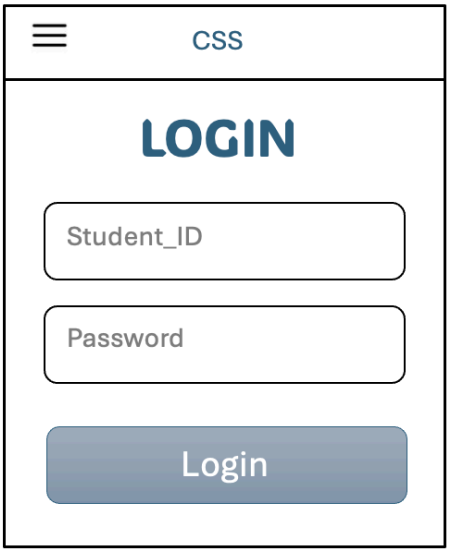
- [ ] HTML/CSS: Build and style resource pages, merchandise order form, order status page, home page, contact page, and admin tables.
- [ ] JavaScript: Validate orders and track resource interactions, implement event filtering and calendar view
- [ ] PHP: Handle resource tracking and merchandise orders
- [ ] Database: Use resources, resource_usage, merch_orders (SELECT, INSERT, UPDATE, DELETE)

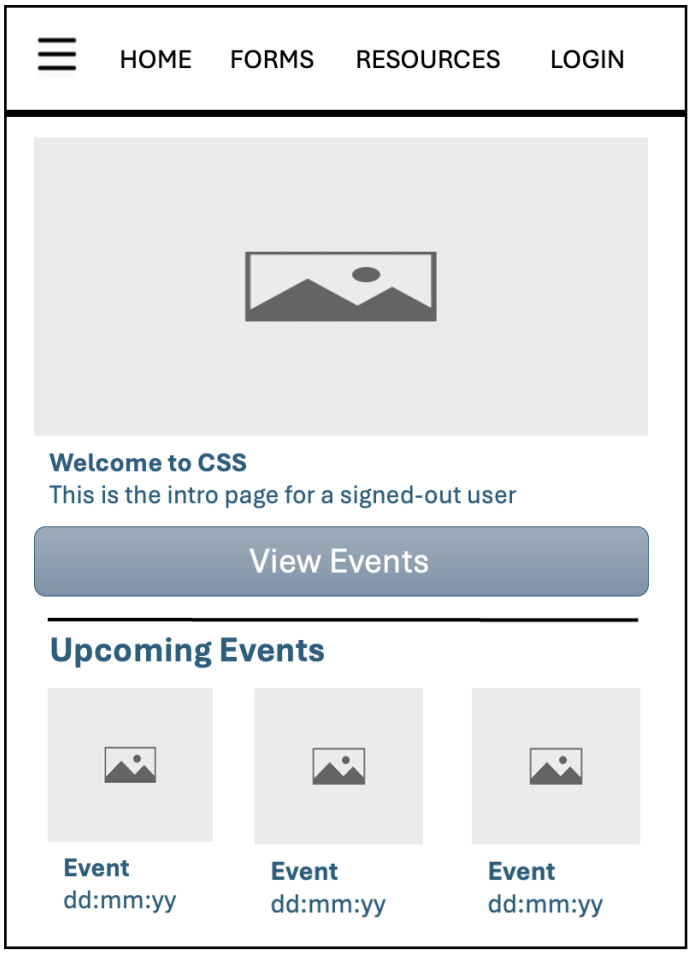
Shared responsibilities

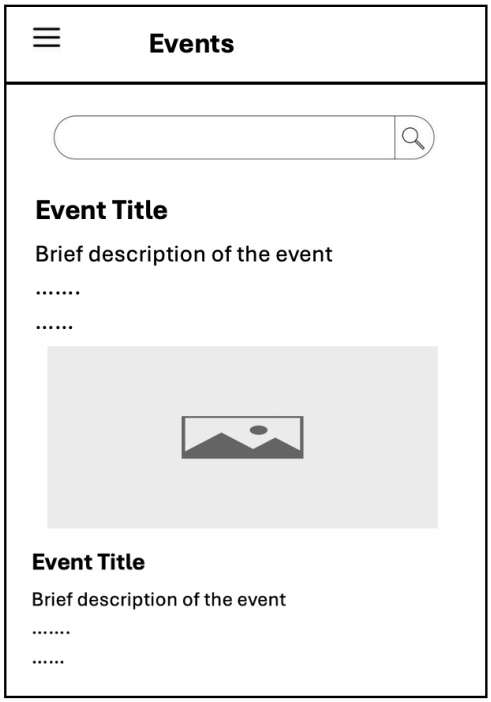
- [ ] Ensure consistent design and responsiveness
- [ ] Integrate all features into one system
- [ ] Test, debug and finalize the application

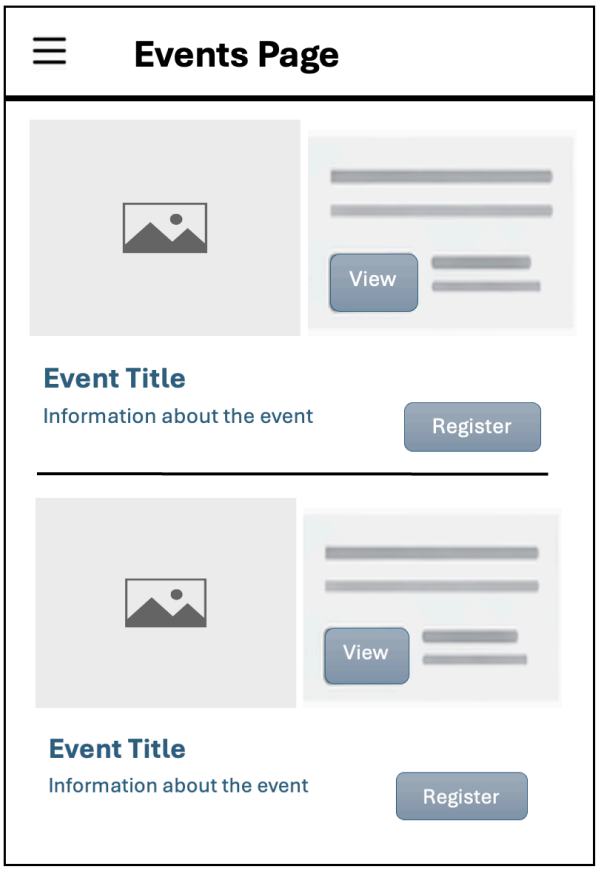
WIREFRAMES

AMANDA - User Accounts & Events

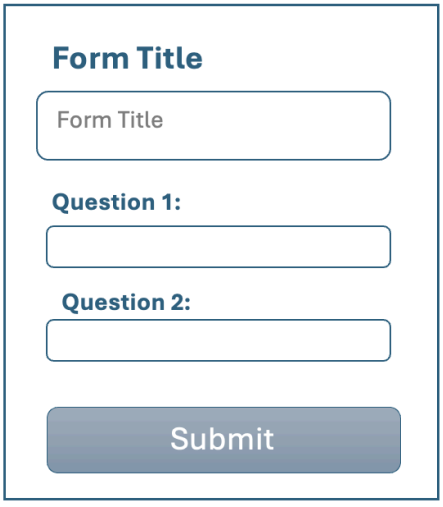


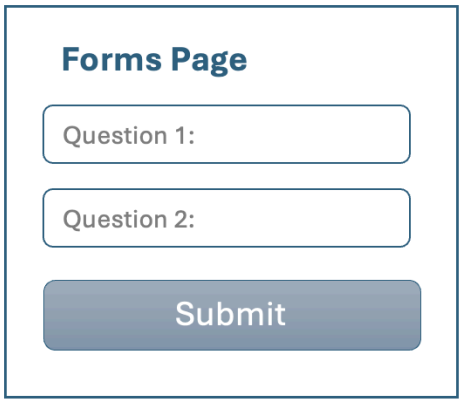


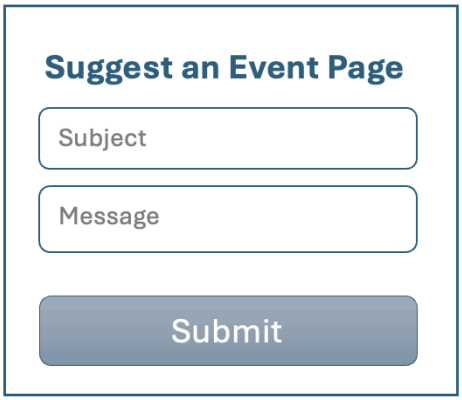




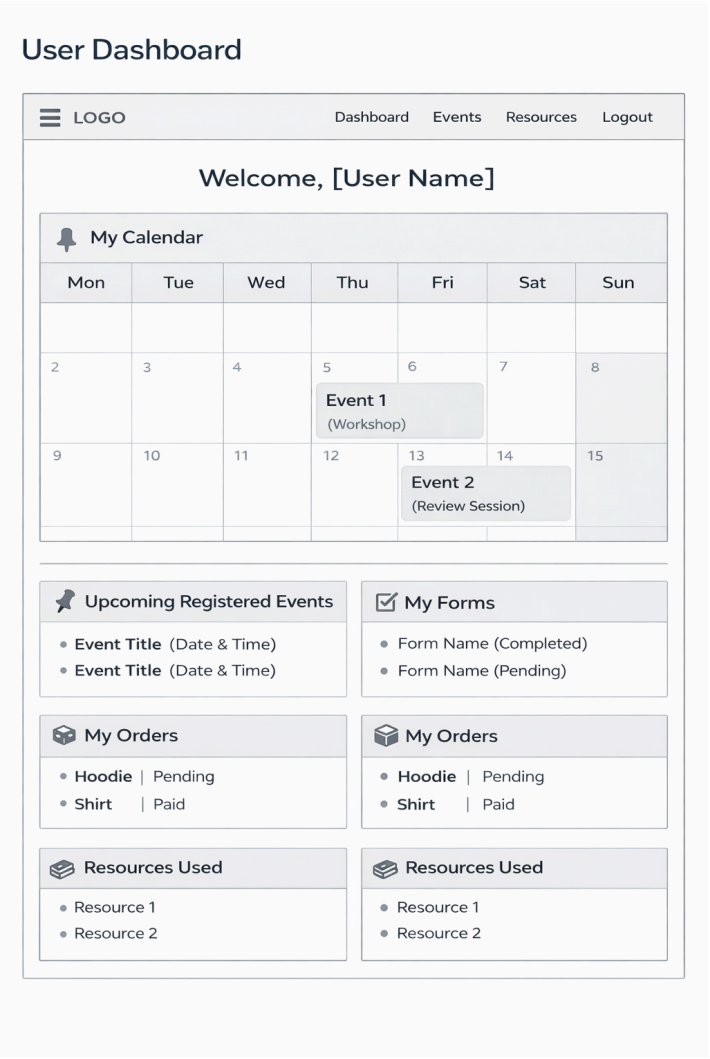
HARSHINI - Forms & Suggestion

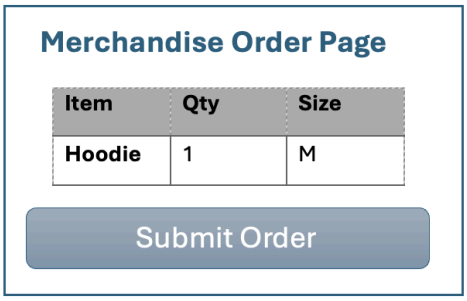


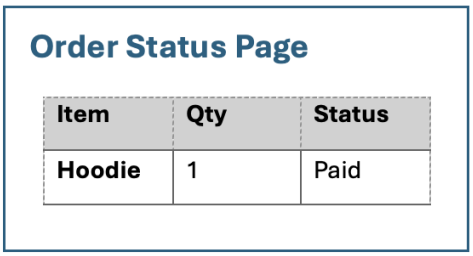




DANIEL - Resource & Merchandise







Project Increments

Increment 1: Basic Structure and Database Setup
Target Date: April 1st - lab 12.2
In this increment, the team will implement the foundational structure of the web application and ensure that the basic layout and database are ready for further development.

Functionality Delivered:

- Users can navigate between core pages (Home, Events, Resources, Login, Forms, Suggestions, Orders, Logout)
- Users can login to their profile, register for events, submit forms and event suggestions, place merch orders, see their dashboard and logout.
- Front and back-end functionality completed
- Database structure is created and ready for use

Tasks and Responsibilities:

- Amanda (User Accounts & Events):
    - Build and implement login and logout
    - Design the events page, registering for events and update the ‘My Registered Events’ table.
    - Begin setting up users, events and event_registrations database tables
- Harshini (Forms & Suggestions):
    - Build forms page and event suggestion pages
    - Create dashboard layout
    - Set up forms, form_questions, form_responses and event_suggestions database tables
- Daniel (Resources & Merchandise):
    - Create the navigation bar and home page layout
    - Build Resources page and Merchandise Order page
    - Create order Status page layout
    - Set up resources, resource_usage and merch_orders database tables

Deliverables:

1. User can login, access information and log out
2. Fully functional events, forms, suggestion and order pages
3. Initial database schema implemented (all tables created)
4. Basic consistency in design across all pages

Client Feedback

Feedback on Development Plan:
During the discussions with the client, Sophie Xiu stated that the design should be *“similar to the Instagram posts”* and use colours such as *“purple, blue and white”*. She also emphasized that the interface should be *“minimalistic, sleek and modern with a simple and easily viewable layout but still consistent with the CSS branding”*.

Changes made:

- Simplified page layouts to reduce clutter and improve readability
- Used clean layouts (eg cards and sections) to improve usability

Ongoing Feedback:
The team will continue to share progress at each project increment to ensure the design and functionality remain aligned with the client’s vision.

Feedback after Increment
After corrections were made to our first increment, we demonstrated an intermediate CSS Web Hub to the client, Sophie Xiu. Overall, she was satisfied with both the functionality and design of the application. She notes that “it looks good” and did not request any additional features or major changes to the system.

The only suggestion provided was related to visual branding. The client specified “you should make the CSS logo more obvious within the website to strengthen its identity” suggesting that we improve the overall branding of the website.

Changes made:

- Added the logo to the navigation bar
- Integrated the logo as a subtle background pattern across the site
- Included a meet the team page

This allows us to reinforce branding while maintaining a clean and non-intrusive user interface.